```python
"""
---------------------------------
File: LinkFamiliesByDemographics.py

Summary: "The Time Machine"
         Links Relational Families across 10-year census gaps using an unbreakable
         10-variable demographic hash (Sex, BirthYr, BPL, FBPL, MBPL) for the couple.
---------------------------------
"""

import os

import duckdb

from python.utils import gen_logging

# ================================================================================
# CONFIGURATION
# ================================================================================
if os.name == 'nt':
    BASE_DATA_DIR = r"D:\Data\Genealogy_Data"
else:
    BASE_DATA_DIR = os.path.expanduser("~/Genealogy_Data")

MASTER_DB = os.path.join(BASE_DATA_DIR, "MasterVault_Relational.db")
MATCH_DB = os.path.join(BASE_DATA_DIR, "DemographicMatches.db")

def link_households_across_decades(logger):
    logger.info("Initializing DuckDB... Finding demographic household matches across ALL consecutive decades.")

    # Check for file locks
    if os.path.exists(MATCH_DB):
        try:
            os.remove(MATCH_DB)
        except PermissionError:
            logger.warning(f"CRITICAL: {MATCH_DB} is locked by another program (likely a DB Viewer).")
            logger.warning("Please close any applications using this database and try again.")
            return

    # DECISION: We use DuckDB here instead of standard SQLite because DuckDB is an OLAP
    # (Online Analytical Processing) engine. It is specifically designed to perform massive,
    # memory-intensive Cartesian joins across millions of rows in seconds.
    con = duckdb.connect()
    con.execute("PRAGMA memory_limit='90GB'")
    con.execute("INSTALL sqlite; LOAD sqlite;")

    logger.info("Attaching Vaults...")
    # DECISION: Read directly from the SQLite Vaults without importing them into Python memory.
    con.execute(f"ATTACH '{MASTER_DB}' AS master (TYPE SQLITE, READ_ONLY);")
    con.execute(f"ATTACH '{MATCH_DB}' AS match_db (TYPE SQLITE);")

    logger.info("Step 1/3: Extracting Head and Spouse Demographics...")
    con.execute("""
        -- DECISION: Pre-compute the 10-Variable Demographic Fingerprint for every couple in the database.
        -- By flattening the Head and Spouse data into a single row per family_id, we make the cross-decade join drastically faster.
        CREATE TEMP TABLE hh_features AS
        SELECT
            f.family_id,
            f.year,
```

```
            h.histid AS head_histid,
            h.sex AS head_sex,
            h.birthyr AS head_birthyr,
            h.bpld AS head_bpld,
            h.fbpl AS head_fbpl,
            h.mbpl AS head_mbpl,
            s.histid AS spouse_histid,
            s.sex AS spouse_sex,
            s.birthyr AS spouse_birthyr,
            s.bpld AS spouse_bpld,
            s.fbpl AS spouse_fbpl,
            s.mbpl AS spouse_mbpl
        FROM master.families f
        JOIN master.individuals h ON f.head_histid = h.histid
        JOIN master.individuals s ON f.spouse_histid = s.histid
        WHERE h.birthyr IS NOT NULL AND s.birthyr IS NOT NULL;
    """)
    feature_cnt = con.execute("SELECT COUNT(*) FROM hh_features").fetchone()[0]
    logger.info(f"  -> Extracted {feature_cnt:,} target couples.")

    logger.info("Step 2/3: Executing 10-Variable Nationwide Demographic Hash...")
    con.execute("""
        -- DECISION: The Nameless Cross-Decade Join.
        -- We link families across time by matching their exact Sex, Own Birthplace (BPLD),
        -- Father's Birthplace (FBPL), Mother's Birthplace (MBPL), and Birth Year (+/- 2).
        -- Notice there is NO reliance on names (First or Last).
        CREATE TEMP TABLE raw_links AS
        SELECT
            y1.year AS year_1,
            y2.year AS year_2,
            y1.family_id AS family_id_1,
            y2.family_id AS family_id_2,
            y1.head_histid AS head_histid_1,
            y2.head_histid AS head_histid_2,
            y1.spouse_histid AS spouse_histid_1,
            y2.spouse_histid AS spouse_histid_2
        FROM hh_features y1
        JOIN hh_features y2
          -- DECISION: Check for 10, 20, and 30-year gaps.
          -- This brilliantly accounts for the 1890 Census Fire (mandatory 20-year gap between 1880 and 1900)
          -- as well as ancestors who might have been traveling or missed by a specific census decade.
          ON y2.year IN (y1.year + 10, y1.year + 20, y1.year + 30)
         AND y1.head_sex = y2.head_sex
         AND y1.spouse_sex = y2.spouse_sex
         AND y1.head_bpld = y2.head_bpld
         AND y1.spouse_bpld = y2.spouse_bpld
         -- DECISION: Birth Year is used instead of Age because Age fluctuates depending on the exact
         -- month the census was taken, whereas Birth Year provides a stable, immutable mathematical anchor.
         AND y2.head_birthyr BETWEEN y1.head_birthyr - 2 AND y1.head_birthyr + 2
         AND y2.spouse_birthyr BETWEEN y1.spouse_birthyr - 2 AND y1.spouse_birthyr + 2
        -- DECISION: Gracefully handle missing parent birthplaces.
        -- If a census taker didn't record a parent's birthplace in one decade, we do not penalize the match.
        WHERE (y1.head_fbpl = y2.head_fbpl OR y1.head_fbpl IS NULL OR y2.head_fbpl IS NULL)
          AND (y1.head_mbpl = y2.head_mbpl OR y1.head_mbpl IS NULL OR y2.head_mbpl IS NULL)
          AND (y1.spouse_fbpl = y2.spouse_fbpl OR y1.spouse_fbpl IS NULL OR y2.spouse_fbpl IS NULL)
          AND (y1.spouse_mbpl = y2.spouse_mbpl OR y1.spouse_mbpl IS NULL OR
```

```
            y2.spouse_mbpl IS NULL);
        """)

        logger.info("Step 3/3: Enforcing Unique 1-to-1 Mathematical Matches...")
        # DECISION: Strict Uniqueness. We enforce COUNT(*) = 1 to guarantee we never accidentally merge twins,
        # cousins with identical demographics, or statistically identical families. If a match isn't 100% unique nationwide, we discard it to protect data integrity.
        con.execute("""
            CREATE TABLE match_db.household_links AS
            SELECT family_id_1, MAX(family_id_2) AS family_id_2,
                   MAX(year_1) AS year_1, MAX(year_2) AS year_2,
                   MAX(head_histid_1) AS head_histid_1, MAX(head_histid_2) AS head_histid_2,
                   MAX(spouse_histid_1) AS spouse_histid_1, MAX(spouse_histid_2) AS spouse_histid_2
            FROM raw_links
            GROUP BY family_id_1
            HAVING COUNT(*) = 1;
        """)

        match_count = con.execute("SELECT COUNT(*) FROM match_db.household_links").fetchone()[0]
        logger.info(f"SUCCESS! Found {match_count:,} perfect multi-decade matches.")
        logger.info(f"Saved to: {MATCH_DB}")


if __name__ == "__main__":
    main_logger = gen_logging.setup_logging(logger_name="DEMO_LINK")
    link_households_across_decades(main_logger)

```